

Introduction

Within this project we are interested in exploring the idea of doing video classification of a set of human movements with resource efficient methods regarding model architecture and dataset size. This idea originated within the work for our study project, where we are planning to do movement recognition, integrated within a Simon Says game that is instructed by a robot. By first exploring resource efficient methods, we will have a chance to experience whether these methods will be enough for precise video classification, and can further be used for the movements we will use in our study project. Therefore, we formulate the following research question: *Is it feasible to achieve high-accuracy movement classification using computationally efficient architectural components?*

We will approach this project from a bottom up perspective and explore increasingly complex models based on previous results.

Methods

Experiment 1 investigated single-frame model configurations, experiment 2 extended this to three frames, and experiment 3 focused on a more complex model with a GRU extension and 8-frame data.

For evaluation, we mostly considered overall accuracy, f1-scores for each class, and confusion matrices.

Dataset

For this project we used parts of the “*Action Recognition Datasets: NTU RGB+D and NTU RGB+D 120*” made available by the ROSE Lab at the Nanyang Technological University, Singapore. The combined datasets contain 120 action classes with 114,480 video samples in total. The samples were captured by three Kinect V2 cameras and are each available as RGB video, depth map sequence, 3D skeletal data, and infrared (IR) video.

Since we wanted to stay close to possible actions used in a Simon Says game and generally available camera equipment, we exclusively used the RGB videos and decided on four action classes: Class 1 Capitulate (A95), Class 2 Clapping (A10), Class 3 Cross Arms (A96) and Class 4 Hand Waving (A23). This reduced our dataset to 4,230 videos with a resolution of 1920x1080 and lengths of approximately 2 seconds. The videos contain different subjects,

locations and angles. Furthermore, the subjects are either standing or sitting. Figure 1 shows an example frame for each of our four action classes.

The subset of the data was downloaded via the *process_NTU.py* file and saved locally on our own machine. Further preprocessing is explained in the respective experiment chapters.

Our Basis - ResNet18

Within this project, we decided to use the pretrained ResNet18 model for transfer learning. This model is part of the ResNet family, which utilizes a deep convolutional neural network architecture featuring residual blocks composed of two convolutional layers and a skip connection, which facilitates the flow of gradients and prevents information loss (He et al., 2015). ResNet18, consisting of 18 layers, is the smallest and therefore the most resource efficient model out of the ResNet family (Writer, 2026). Therefore, it was the most suitable version for our purpose. The ResNet18 architecture can be seen in Figure 2.

We utilized a ResNet18 architecture with weights pretrained on ImageNet. The capacity of the model therefore already includes pre-learned representations of 1000 different object categories, including the recognition of humans. (*Resnet18 — Torchvision Main Documentation*, o. D.)

To stay consistent, we reshaped our image input size to 224x224 across all experiments.

Experiment 1: Single-Frame-Models

For experiment 1 we extracted one single frame from each video and splitted the dataset into train, test and validation data with a 70-15-15 split. The code for this process can be found in the *extract_frame.py* file.

For the first model we used the frame at the three-quarter-position from the videos. Model 1a used the pretrained ResNet18 with the default weights and a modified last layer to output 4 classes. During training only the last classification layer was updated while the ResNet18 layers were frozen. A detailed description of the hyperparameter choices can be found in Table 1.

We used Optuna for hyperparameter optimization on Model 1a, resulting in Model 1b. This part of experiment 1 involved 10 trials with the search extending to different values for learning rate (*1e-5 to 1e-2*), weight decay (*1e-6 to 1e-2*), momentum (*0.8 to 0.99*), and optimizer (*SGD and Adam*).

The resulting Model1b, was furthermore used as a basis for Model 1c, which additionally had layer 4 of the ResNet18 unfrozen.

Overall, we trained three versions of the ResNet18 model for experiment 1, which all used the single three-quarter-frame data.

Experiment 2: Combined Three-Frame-Approach

Model 2 and 3 both used the architecture version of Model 1b. However, Model 2 was trained on the frame that was sampled from the middle of each video, whereas Model 3 got the last frame of each video as input.

Furthermore, we combined these models into Model 4. Within this model, each submodel received their respected sampled frame as input data. The predicted class probabilities of each of the submodels were averaged, resulting in one final prediction.

Experiment 3: Gated Recurrent Unit

For experiment 3 we extracted eight frames from each video and splitted the dataset into train, test and validation data with a 70-15-15 split. The code for this process can be found in the *extract_videos.py* file.

Within experiment 3, we decided to integrate a Gated Recurrent Unit (GRU) layer with the frozen ResNet18 model. Thus, each sampled frame was passed through the frozen ResNet18 for feature extraction. Afterwards, The GRU layer was used for sequential processing. Finally, a fully connected (FC) linear layer classified the resulting sequence representation into one of the four classes.

In the first model of this experiment, Model 5a, only the GRU and FC layers were trained. A detailed description of the hyperparameter choices can be found in Table 1.

Next, we tuned the hyperparameters of Model 5a. The resulting optimised settings were used to create Model 5b. Once again, the detailed description of the hyperparameter choices can be found in Table 1.

Finally, for Model 5c, we used the optimized hyperparameter choices from Model 5b and additionally introduced temporal differences by subtracting the features of consecutive frames, giving the GRU a different type of input that focuses more on the change over time.

Results & Analysis

A detailed summary of the results of all models can be found in Table 2.

The first model in experiment 1, Model 1a, resulted in an overall accuracy of 0.67. Further evaluation showed that class 1 (capitulate) performed much better than the other classes.

With hyperparameter optimization overall improvement was minimal as the accuracy of Model 1b slightly rose to 0.68. Unfreezing layer 4 of the ResNet18 in Model 1c illustrated the greatest improvement with an increase in the overall accuracy to 0.78 and a more balanced confusion matrix across classes.

Overall, experiment 1 showed that class 1 reached nearly perfect classification across all three models, while classification of the other classes exhibited worse performances. Furthermore, unfreezing part of the ResNet18 improved performance but also required more training time and computational capacity.

For experiment 2 we chose to work with the architecture of Model 1b, since it was the better option out of the less computationally expensive models.

The results of Model 2 and Model 3 showed that the predictions for class 1 classification had nearly perfect results across the different time stamps of the videos, while performance across the other classes varied, as can be seen in Figure 3.

Model4 resulted in an accuracy of 0.74 with class 1 being perfectly predicted. The model especially struggled with class 2 (clapping) with an f1-score of 0.58.

All in all, experiment 2 illustrated different performance based on the type of the class and the sampled point in time. This behaviour was only slightly balanced out by Model 4.

Within experiment 3, our first model, Model 5a, utilized the ResNet18 + GRU architecture and achieved an overall accuracy of 0.6. We then implemented hyperparameter tuning to create Model 5b, hoping to improve the models output. This slightly increased the overall accuracy to 0.63, but still showed large performance gaps across classes. The confusion matrix and f1-scores showed that Model 5b especially struggled with classes 2 (clapping) and 4 (waving). In the subsequent model, Model 5c, we used the additional temporal differentiation. This approach improved the prediction accuracy for individual classes, with an overall accuracy of 0.66.

Figure 4 shows the overall accuracies of all models.

Discussion & Interpretation

In experiment 1 we found that using only a single frame and a very basic set-up is enough to achieve a moderate accuracy in classifying our four classes. While unfreezing the fourth layer of the ResNet18 improved classification the most, we decided to focus on investigating methods that do not require fine-tuning of the ResNet18 to save time and computational costs.

Our most significant take-away from Model 1b, 2, and 3 is the connection between the sampled point in time and class performances. This is likely due to the timing of the different

classes in the dataset. For example, taking the last frame of the video leads to the best performance since some movements might not have started at the middle or three-quarter mark and therefore no significant behavior is reflected (see Figure 5). Furthermore, class 2 (clapping) might be more difficult to classify due to the arm position being close to the body and therefore less visible than the extended arms. Interestingly, classification of class 1 (capitulate) achieved almost perfect results across all models. Analyzing the data revealed that this is the only class with outside backgrounds. While this might improve robustness of the class, it probably does not explain this huge performance gap between the classes. Therefore, another possibility might be that class 1 has the most unique arm position out of the four classes, so it is less likely to be confused with other more natural movements.

Model 4 showed that combining the different time frames and models increased performance slightly, leading us to assume that a time-sensitive approach might further improve classification.

Therefore, we decided to use sequential data from our video dataset as input for our next models. To handle the temporal nature of this data, we extended the ResNet18 architecture by adding a GRU layer. Given the short length of our video clips and the need for computational efficiency, we chose the GRU as it provides a lightweight alternative to the LSTM (Srivatsavaya, 2021). Since our video clips were brief, we limited the sampling to eight frames per video to maintain efficiency and comply with our available resources.

The main takeaway from the results of the third experiment is that a more complex model structure did not improve the overall accuracy compared to our previous approaches. Even with the tuned hyperparameters of Model 5b and the modified input features of Model 5c, we did not see an increase in the overall accuracy of the models. This indicates that this specific architecture may not be the best fit for our data.

Although accuracy for class 1 remained high, the other classes did not show drastic improvements. While changing the feature input made the class predictions of class 2 - 4 more precise, the impact of the overall accuracy was minimal. We suspect that the number of frames sampled may be insufficient. Additionally, the models showed signs of overfitting during training with very high training accuracies and lower validation accuracies, suggesting that the model is too complex for the size of our dataset. Future improvements would likely require either a different modeling approach or more input data.

Conclusion & Future Outlook

In this project we were interested in exploring the following research question: *Is it feasible to achieve high-accuracy movement classification using computationally efficient architectural components?*

Overall, the results of our different experiments indicate that our chosen resource efficient approaches can achieve moderate movement classification accuracy. Thus, we can conclude that it is not feasible to achieve high-accuracy movement classification with computationally efficient architectural components. However, increasing model complexity, which goes hand in hand with more computational cost, did not improve the performance of the overall accuracy. This suggests that either more sequential input data or different, more computationally expensive, architectural components may be needed. Using a partially or fully unfreezed ResNet18 for finetuning seems to be a promising next step, as illustrated by the increase in overall accuracy of Model 1c.

Moreover, further research could also include using the skeletal or depth data included in the ROSE dataset. Regarding the different architectural components, it might be beneficial to use a small transformer model or attention layers as future approaches to determine whether these more computationally expensive approaches yield better results.

Since we were also interested in whether our trained models could predict our own videos featuring the movements from the various classes, and whether this could thus serve as an initial baseline for our Simon Says movement classification, we used the best-performing model to see how it would classify these videos. Therefore we used Model 4. Its predictions are lacking ...

References

- Abhi, I. (2023, October 24). *Fine-tuning a pre-trained ResNet-18 model for image classification on custom dataset with PyTorch*. Medium.
<https://medium.com/@imabhi1216/fine-tuning-a-pre-trained-resnet-18-model-for-image-classification-on-custom-dataset-with-pytorch-02df12e83c2c>
- He, K., Zhang, X., Ren, S. & Sun, J. (2015, 10. Dezember). *Deep Residual Learning for Image Recognition*. arXiv.org. <https://arxiv.org/abs/1512.03385>
- Ou, X., Yan, P., Zhang, Y., Tu, B., Zhang, G., Wu, J., & Li, W. (2019). Moving object detection method via RESNET-18 with Encoder–Decoder structure in complex scenes. *IEEE Access*, 7, 108152–108160.
<https://doi.org/10.1109/access.2019.2931922>
- resnet18* — *Torchvision main documentation*. (o. D.).
<https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18>
- Srivatsavaya, P. R. (2021, August 12). *LSTM vs GRU*. Medium.
<https://medium.com/@prudhviraju.srivatsavaya/lstm-vs-gru-c1209b8ecb5a>
- Writer, C. (2026, 31. Juli). *What Is ResNet-18? How to Use the Lightweight CNN Model*. Roboflow Blog. <https://blog.roboflow.com/resnet-18/>

Appendix

The whole project can be found on: https://github.com/mschl7/action_recognition.git



Figure 1: Example frames from the four action classes (from left-right, top-down): Capitulate, Clapping, Cross Arms and Hand Waving.

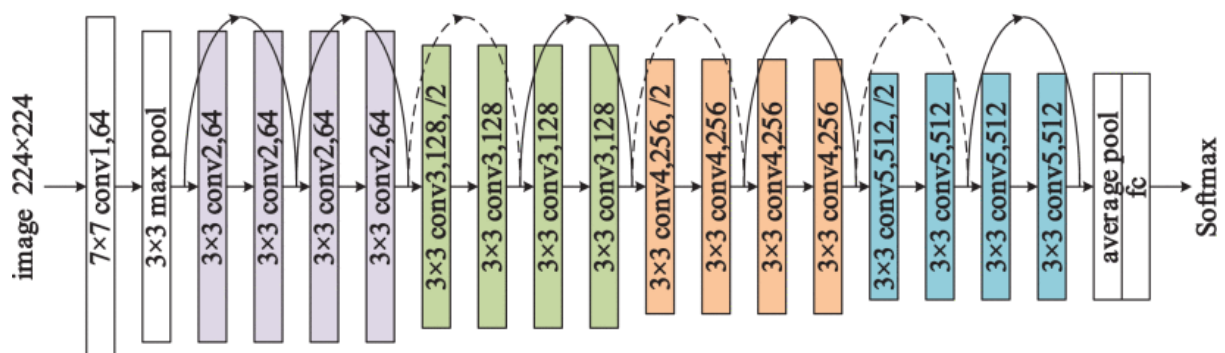
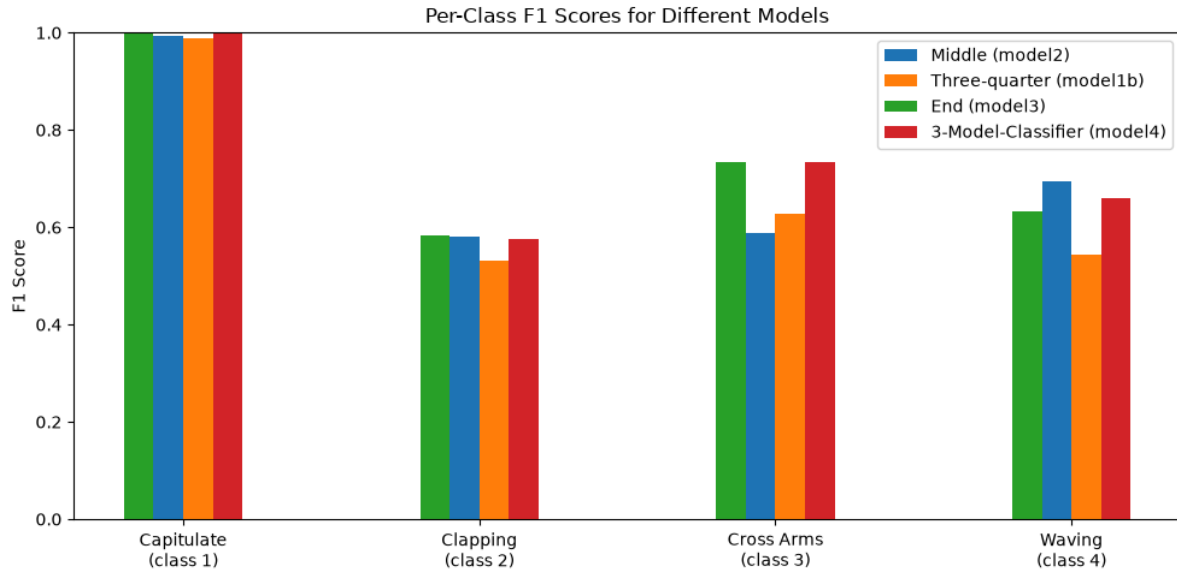
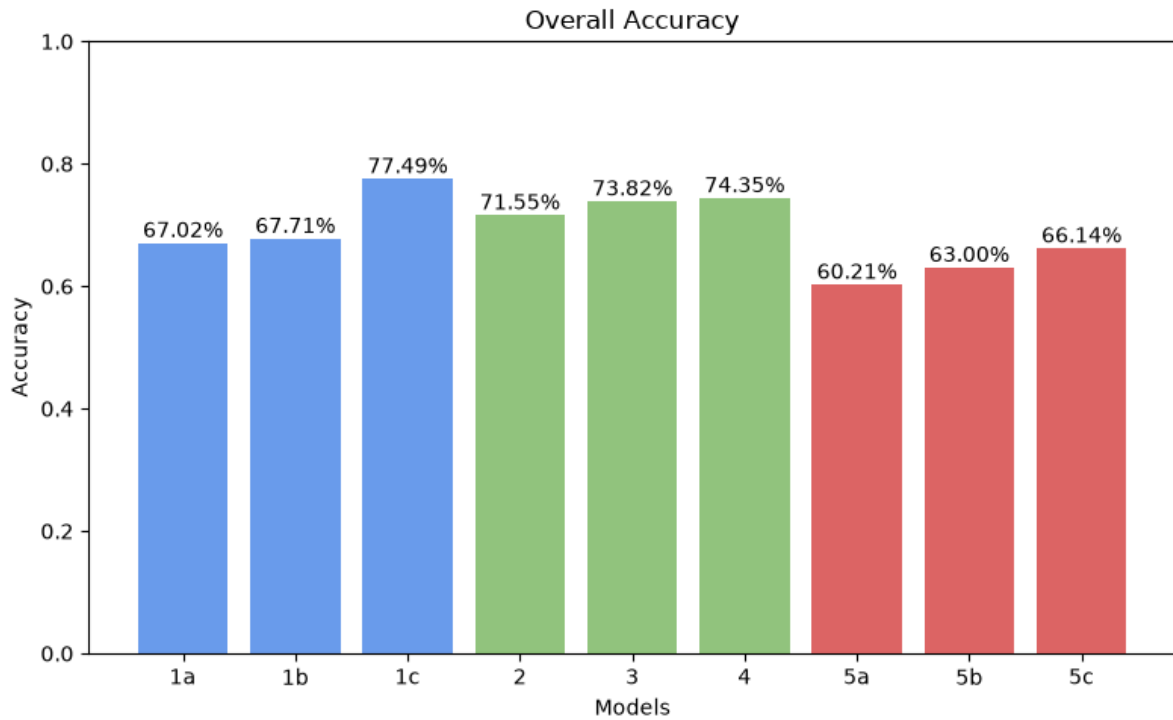
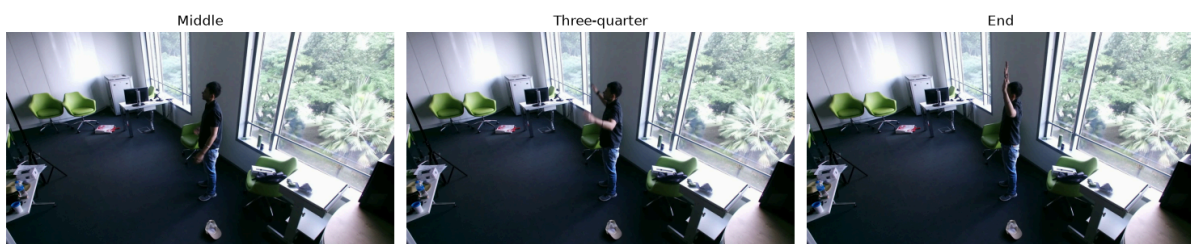


Figure 2: ResNet18 Architecture (Ou et al., 2019)

Figure 3: *Per-Class F1-Scores for experiment 1 and 2.*Figure 4: *Overall Accuracies of all models, with Experiment 1 in blue, Experiment 2 in green, and Experiment 3 in red.*Figure 5: *Example of different movements based on time-stamp.*

Model	Description	Data	Hyperparameter
Model1a	Frozen ResNet18 + modified last layer to 4 classes	frame at $\frac{3}{4}$	Optim: SGD Lr: 0.001 Momentum: 0.9 Epochs: 10
Model1b	Model1a with best Hyperparameters	frame at $\frac{3}{4}$	Optim: Adam Lr: $\sim 3e-3$ Weight Decay: $\sim 1e-3$ Epochs: 10
Model1c	Model1b + unfrozen ResNet18 Layer 4	frame at $\frac{3}{4}$	Optim: Adam Lr: $\sim 3e-3$ Weight Decay: $\sim 1e-3$ Epochs: 10 Layer 4: Lr: $1e-5$
Model2	Model1b with different data	frame at $\frac{1}{2}$	Optim: Adam Lr: $\sim 3e-3$ Weight Decay: $\sim 1e-3$ Epochs: 10
Model3	Model1b with different data	frame at end	Optim: Adam Lr: $\sim 3e-3$ Weight Decay: $\sim 1e-3$ Epochs: 10
Model4	Combined averaged predictions from Model1b, Model2, and Model3	frames at $\frac{1}{2}$, $\frac{3}{4}$, and end	
Model5a:	frozen ResNet18 + GRU layer + linear Classifier layer	8 frames per video	Optim: Adam Lr: 0.001 Epochs: 10
Model5b	frozen ResNet18 + GRU layer + linear Classifier layer + tuned Hyperparameters	8 frames per video	Optim: Adam Lr_GRU: 0.00079 Lr_FC: 0.00058 Weight_Decay_GRU: 0.0000172 Weight_Decay_FC: 0.0000100 Epochs: 10

Model5c	frozen ResNet18 + GRU layer + linear Classifier layer + tuned Hyperparameters + temporal differencing	8 frames per video	Optim: Adam Lr_GRU: 0.00079 Lr_FC: 0.00058 Weight_Decay_GRU: 0.0000172 Weight_Decay_FC: 0.0000100 Epochs: 10
---------	---	--------------------	---

Table 1: Summary of all model configurations with Experiment 1 in blue, Experiment 2 in green, and Experiment 3 in red.

Model	Accuracy	F1-Score Class 1	F1-Score Class 2	F1-Score Class 3	F1-Score Class 4
1a	0.670	0.983	0.529	0.631	0.510
1b	0.677	0.990	0.531	0.628	0.544
1c	0.775	1.000	0.651	0.763	0.686
2	0.716	0.993	0.580	0.588	0.693
3	0.738	1.000	0.584	0.734	0.633
4	0.743	1.000	0.576	0.733	0.660
5a	0.602	1.000	0.455	0.392	0.527
5b	0.630	1.000	0.343	0.605	0.487
5c	0.693	0.841	0.635	0.678	0.626

Table 2: Summary of overall accuracy and per-class F1-score for all models with Experiment 1 in blue, Experiment 2 in green, and Experiment 3 in red.